



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

ROBOTY MOBILNE

Instrukcja 1

Kinematyka robota mobilnego



12 kwietnia 2025

Spis treści

1	Wprowadzenie	3
1.1	Transformacje	3
1.2	Ograniczenia kinematyczne	4
1.2.1	Koło stałe (Fixed standard wheel)	4
1.2.2	Koło skretne (Steered standard wheel)	5
1.2.3	Koło omni (Omnidirectional wheel)	5
1.3	Złożenie ograniczeń wszystkich kół	6
1.4	Kinematyka prosta	7
1.5	Kinematyka odwrotna	7
1.6	Dlaczego używamy pseudoinwersji macierzy	7
1.6.1	Mecanum	7
1.6.2	Car-like (Ackermann)	8
1.6.3	Skid-steer	10
1.7	Obliczanie kąta skrętu koła (steering angle)	11
1.7.1	Czysta translacja	11
1.7.2	Czysty obrót	11
1.7.3	Połączenie translacji i obrotu	11
1.7.4	Korekta kąta dla ruchu wstecznego	12
1.8	Analiza mobilności i sterowalności robota mobilnego	12
1.8.1	Stopnie swobody robota (Degrees of Freedom, DoF)	12
1.8.2	Mobilność robota (Degree of Mobility, δ_m)	12
1.9	Sterowalność robota (Degree of Steerability, δ_s)	13
1.9.1	Typowe wartości:	13
1.10	Manewrowalność robota (Degree of Maneuverability, δ_M)	13
1.10.1	Na przykład:	13
2	Roboty	14
2.1	Stanowisko 1 - napęd różnicowy	14
2.2	Stanowisko 2 - napęd skid-steer	15
2.3	Stanowisko 3 - napęd car-like	16
2.4	Stanowisko 4 - napęd omni (swerve drive)	17
2.5	Stanowisko 5 - napęd omni (mecanum)	18
2.6	Uruchomienie robota	19
2.7	Uruchomienie środowiska Jupyter Notebook	21
2.8	Sterowanie robotem	21
2.9	Wizualizowanie wyników z rviz2	23
3	Program ćwiczenia	24

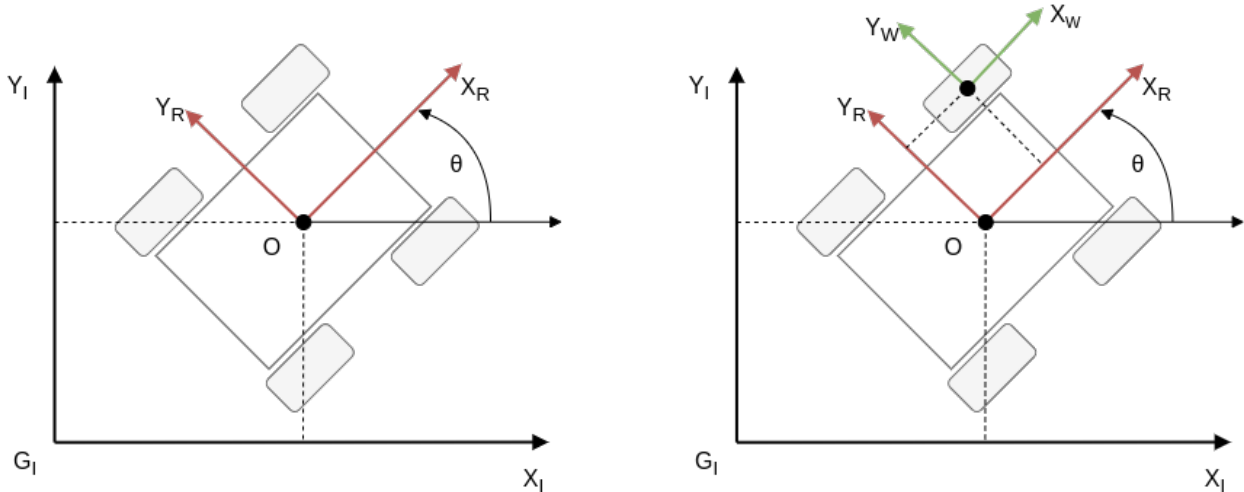
1 Wprowadzenie

WSZYSTKIE PLIKI Z ZADANIAM I PRZYKŁADY SĄ W FORMACIE .ipynb studenci pobierają plik z zadaniami i mogą go modyfikować. Na dockerze jest gotowy katalog, który można skopiować bez konieczności pobierania z wikampa W instrukcjach dopisane odpowiednie uruchomienie (symulacja lub rzeczywisty robot -> w jednym terminalu odpala się albo wszystko to co jest niezbędne do uruchomienia symulacji albo robota.)

1.1 Transformacje

Budowanie modelu kinematycznego robota mobilnego należy rozpocząć od zrozumienia wpływu każdego z kół na możliwości poruszania się robota. Każde koło w inny sposób przyczynia się do przeniesienia swojej ruch obrotowego na ruch platformy. Ma to związek z jego umiejscowieniem, rozmiarem oraz typem. Każde z kół wprowadza do modelu pewne ograniczenia kinematyczne, których superpozycja definiuje mobilność całego robota.

Na zajęciach zajmujemy się ruchem robota na płaszczyźnie, stąd pozycja robota jest określona trzema stopniami swobody - położeniem xy , oraz orientacją θ wokół osi Z .



Rys. 1.1: Graficzne przedstawienie robota na płaszczyźnie z zaznaczeniem układów współrzędnych związanych z podłożem, robotem i pojedynczym kołem.

Układ $O : X_R, Y_R$ jest związany z platformą robota. Zwrot osi X_R jest zgodny z kierunkiem jazdy do przodu a początek układu współrzędnych robota umieszczony jest w punkcie, który najczęściej znajduje się pomiędzy kołami napędowymi i stanowi punkt referencyjny dla określania pozycji kół. Aby określić pozycję robota w przestrzeni należy układ robota zrzutować na globalny układ $G : X_I, Y_I$.

Pozycję robota w inercyjnym układzie G można przedstawić za pomocą wektora 1.1.

$$\xi_I = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (1.1)$$

W celu transformacji między lokalnym układem robota O a inercyjnym, globalnym układem G wykorzystamy macierz rotacji wokół osi Z 1.2. Ta transformacja jest przydatna np. w celu przeliczenia składowych prędkości z układu globalnego do układu robota 1.3.

$$R(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (1.2)$$

gdzie θ to orientacja robota na płaszczyźnie.

$$\dot{\xi}_R = R(\theta)\dot{\xi}_I \quad (1.3)$$

1.2 Ograniczenia kinematyczne

W celu stworzenia modelu kinematycznego robota konieczne jest uwzględnienie ograniczeń na ruch jakie wprowadza każde koło z osobna.

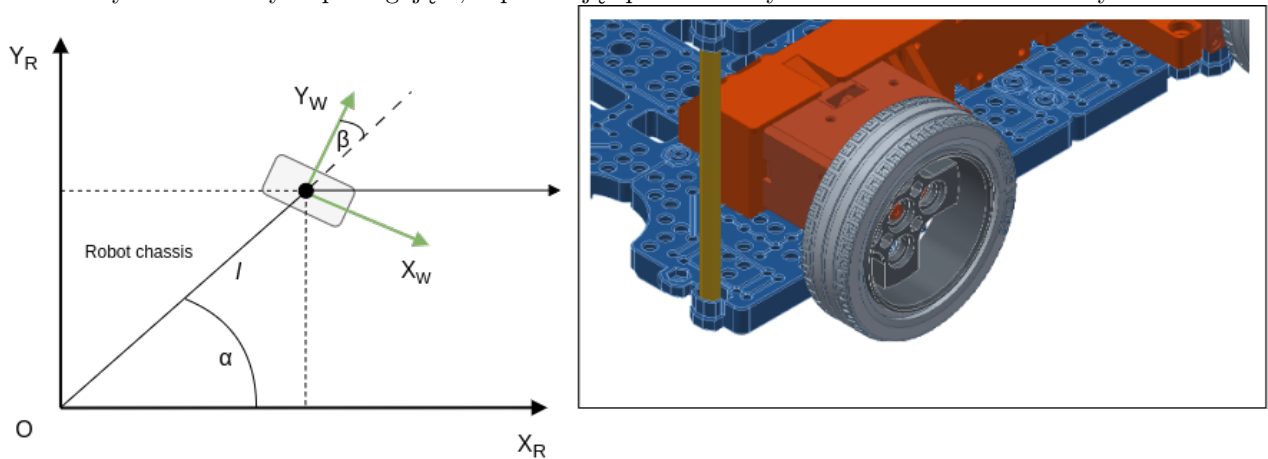
W celu uproszczenia rozważań należy dokonać dwóch założeń:

- Wektor prędkości postępowej koła (zaczepiony w punkcie obrotu koła, np. oś) jest zawsze skierowany równolegle do podłoża. Koło i podłoże mają dokładnie jeden punkt kontaktu i w tym punkcie nie występuje poślizg poprzeczny. (No-sliding constraint)
- Koło obraca się bez poślizgu wzdłużnego (rolling without slipping). Inaczej mówiąc - gdy następuje ruch robota koło musi się obracać w odpowiednim kierunku. (Rolling constraint)

Złożenie ograniczeń poszczególnych kół pozwala stworzyć model kinematyczny całego pojazdu. Poniżej opis trzech podstawowych typów kół i ich charakterystyka.

1.2.1 Koło stałe (Fixed standard wheel)

Koło na stałe przytwierdzone do korpusu robota. Ma jedynie jeden stopień swobody, pozwalający na ruch wzdłuż linii prostej zgodnie z jego orientacją. Nie umożliwia ono skręcania, co oznacza, że kierunek ruchu jest zdeterminowany przez stałą orientację osi koła. Koła te są często stosowane w robotyce mobilnej jako stabilizatory lub elementy wspomagające, zapewniając prostoliniowy ruch bez możliwości zmiany kierunku.



Rys. 1.2: Graficzne przedstawienie koła stałego w układzie robota.

Warunek na czyste toczenie (pure rolling constraint):

$$\begin{bmatrix} \sin(\alpha + \beta) & -\cos(\alpha + \beta) & (-l)\cos(\beta) \end{bmatrix} \dot{\xi}_R - r\dot{\phi} = 0 \quad (1.4)$$

gdzie:

- $\dot{\varphi}$ prędkość obrotowa koła ($\frac{rad}{s}$)
- r promień koła
- l odległość koła od wybranego punktu zaczepienia układu robota na korpusie.

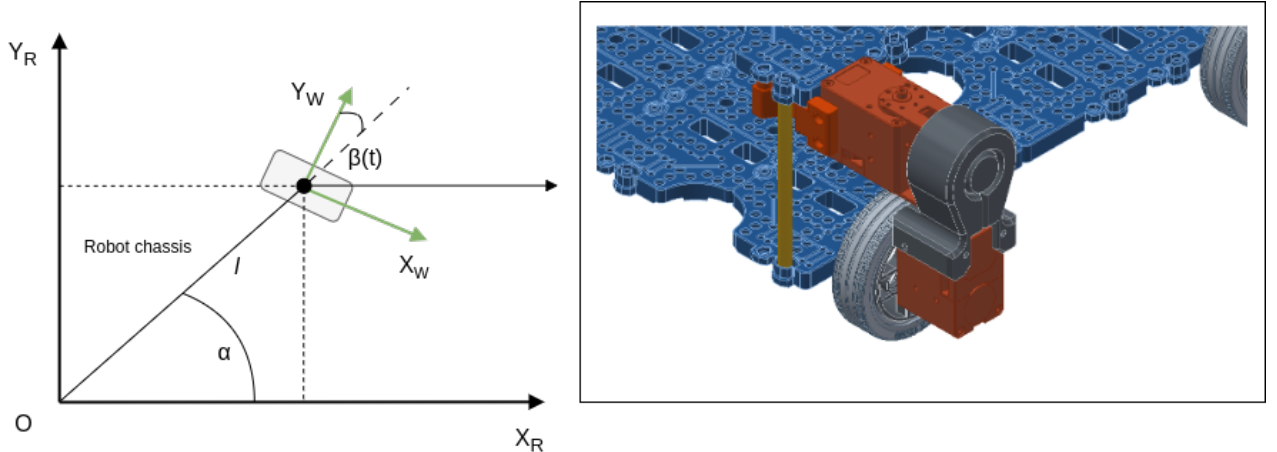
Wektor trzelementowy na początku równania określa w jaki sposób składowe ruchu $\dot{x}, \dot{y}, \dot{\theta}$ przekładają się na ruch w płaszczyźnie koła. Ten ruch zgodnie z założeniem czystego toczenia musi być równy $\dot{\varphi}r$ czyli dokładnie tyle ile wynika z obrotu koła.

Warunek na brak poślizgu poprzecznego (sliding constraint) 1.5 zapewnia, że ruch koła prostopadły do jego płaszczyzny wynosi zero (nie ma innego ruchu w punkcie styku koła z podłożem niż toczenie):

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l\sin(\beta) \end{bmatrix} \dot{\xi}_R = 0 \quad (1.5)$$

1.2.2 Koło skrętne (Steered standard wheel)

Koło skrętne to koło, które posiada dwa stopnie swobody, umożliwiające zarówno ruch wzdłuż linii prostej, jak i zmianę orientacji poprzez skręt. Dzięki możliwości sterowania kątem skrętu, koło to pozwala na elastyczne poruszanie się robota w różnych kierunkach. Koła skrętne są szeroko stosowane w pojazdach i robotach mobilnych, gdzie wymagana jest precyzyjna kontrola nad kierunkiem ruchu.



Rys. 1.3: Graficzne przedstawienie koła skrętnego w układzie robota.

Ograniczenia na toczenie i poślizg:

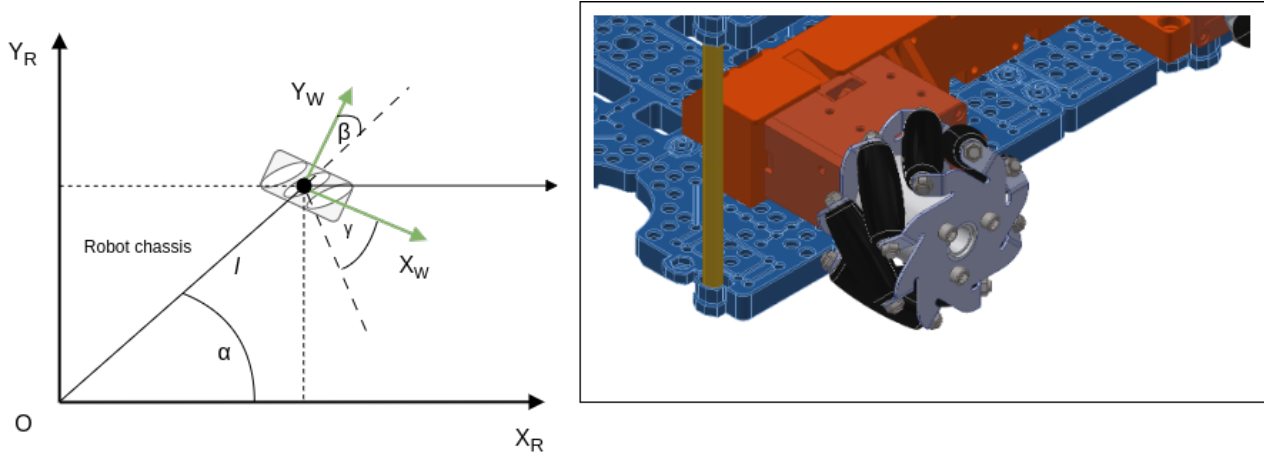
$$\begin{bmatrix} \sin(\alpha + \beta) & -\cos(\alpha + \beta) & (-l)\cos(\beta) \end{bmatrix} \dot{\xi}_R - r\dot{\varphi} = 0 \quad (1.6)$$

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l\sin(\beta) \end{bmatrix} \dot{\xi}_R = 0 \quad (1.7)$$

1.2.3 Koło omni (Omnidirectional wheel)

Koło omni to koło posiadające trzy stopnie swobody, co umożliwia ruch w dowolnym kierunku bez konieczności zmiany orientacji koła. Dzięki zestawowi małych rolek umieszczonych wokół obwodu koła, koło omni może poruszać się zarówno wzdłuż swojej osi, jak i prostopadle do niej. Koła tego typu są często wykorzystywane w

robotyce mobilnej, gdzie wymagana jest duża zwrotność i możliwość ruchu w wielu kierunkach bez skręcania całego robota.



Rys. 1.4: Graficzne przedstawienie koła omni w układzie robota.

Ograniczenia na toczenie i poślizg:

$$\begin{bmatrix} \sin(\alpha + \beta + \gamma) & -\cos(\alpha + \beta + \gamma) & (-l)\cos(\beta + \gamma) \end{bmatrix} \dot{\xi}_R - r\dot{\varphi}\cos(\gamma) = 0 \quad (1.8)$$

$$\begin{bmatrix} \cos(\alpha + \beta + \gamma) & \sin(\alpha + \beta + \gamma) & l\sin(\beta + \gamma) \end{bmatrix} \dot{\xi}_R - r\dot{\varphi}\sin(\gamma) - r_{roller}\dot{\varphi}_{roller} = 0 \quad (1.9)$$

gdzie:

- φ_{roller} prędkość obrotowa pasywnego rollera
- r_{roller} promień pasywnego rollera
- γ kąt ustawienia rollera względem osi X koła

1.3 Złożenie ograniczeń wszystkich kół

Zakładając robota o n kołach stałych lub skrętnych możemy połączyć wszystkie ograniczenia w postaci macierzowej. Ograniczenia na brak poślizgu wprowadzają tylko koła stałe i skrętne ponieważ one mogą toczyć się tylko wzdłuż osi X układu koła.

$$\begin{bmatrix} J_{1f} \\ J_{1s}(\beta_s) \\ C_{1f} \\ C_{1s}(\beta_s) \end{bmatrix} \dot{\xi}_R = \begin{bmatrix} J_2 \\ 0 \end{bmatrix} \dot{\varphi} \quad (1.10)$$

$$A = \begin{bmatrix} J_{1f} \\ J_{1s}(\beta_s) \\ C_{1f} \\ C_{1s}(\beta_s) \end{bmatrix}, B = \begin{bmatrix} J_2 \\ 0 \end{bmatrix} \quad (1.11)$$

gdzie:

- J_{1f} ograniczenia na toczenie dla kół stałych

- $J_{1s}(\beta_s)$ ograniczenia na toczenie dla kół skrętnych z podstawionym znanym kątem skrętu β_s dla danego koła
- C_{1f} ograniczenia na brak poślizgu dla kół stałych
- $C_{1s}(\beta_s)$ ograniczenia na brak poślizgu dla kół skrętnych z podstawionym znanym kątem skrętu β_s dla danego koła
- $J_2 = \text{diag}(r_1, \dots, r_n)$ macierz diagonalna zawierająca promienie kół

$$A\dot{\xi}_R = B\dot{\varphi} \quad (1.12)$$

Przekształcając 1.12 otrzymamy równania na obliczenie $\dot{\xi}_R$ oraz $\dot{\varphi}$.

1.4 Kinematyka prosta

Czyli w jaki sposób zmierzona prędkość kół przekłada się na prędkość robota?

$$\dot{\xi}_R = (A^T A)^{-1} A^T B \dot{\varphi} \quad (1.13)$$

1.5 Kinematyka odwrotna

Czyli w jaki sposób zadana prędkość robota przekłada się na prędkości kół?

$$\dot{\varphi} = (B^T B)^{-1} B^T A \dot{\xi}_R \quad (1.14)$$

1.6 Dlaczego używamy pseudoinwersji macierzy

Spójrzmy na złożenia ograniczeń kinematycznych dla trzech różnych platform:

- Mecanum,
- Car-like (Ackermann),
- Skid steer.

1.6.1 Mecanum

Układ z kołami mecanum może poruszać się z poprzeczną prędkością v_y , wobec tego nie spełnia ograniczeń na brak poślizgu bocznego - tych równań nie uwzględniamy w macierzy $\mathbf{A}$.

Tab. 1.1: Parametry kół robota mecanum

Koło	Położenie	Rodzaj	Ogr. toczenia	Ogr. poślizgu
1	lewy przód	mecanum	tak	nie
2	lewy tył	mecanum	tak	nie
3	prawy tył	mecanum	tak	nie
4	prawy przód	mecanum	tak	nie

$$A = \begin{bmatrix} \sin(\alpha_1 + \beta_1 + \gamma_1) & -\cos(\alpha_1 + \beta_1 + \gamma_1) & -l_1 \cos(\beta_1 + \gamma_1) \\ \sin(\alpha_2 + \beta_2 + \gamma_2) & -\cos(\alpha_2 + \beta_2 + \gamma_2) & -l_2 \cos(\beta_2 + \gamma_2) \\ \sin(\alpha_3 + \beta_3 + \gamma_3) & -\cos(\alpha_3 + \beta_3 + \gamma_3) & -l_3 \cos(\beta_3 + \gamma_3) \\ \sin(\alpha_4 + \beta_4 + \gamma_4) & -\cos(\alpha_4 + \beta_4 + \gamma_4) & -l_4 \cos(\beta_4 + \gamma_4) \end{bmatrix}$$

gdzie:

- α_n – orientacja koła n w układzie robota,
- β_n – kąt skrętu koła (dla kół stałych $\beta_n = 0$),
- γ_n – orientacja rolki na kole mecanum,
- l_n – odległość koła od środka układu robota.

$$B = \begin{bmatrix} r \cos(\gamma_1) & 0 & 0 & 0 \\ 0 & r \cos(\gamma_2) & 0 & 0 \\ 0 & 0 & r \cos(\gamma_3) & 0 \\ 0 & 0 & 0 & r \cos(\gamma_4) \end{bmatrix}$$

Gdzie:

- r – promień koła,
- γ_n – orientacja rolki na kole mecanum.

Przy wyliczaniu wektora prędkości kół ($\dot{\varphi}$) rozwiązujemy układ czterech równań w celu wyliczenia czterech niewiadomych wartości prędkości. Możemy zastosować wzór:

$$\dot{\varphi} = B^{-1} A \dot{\xi}_R \quad (1.15)$$

ponieważ macierz **B** jest macierzą kwadratową oraz ma niezerowe wartości na diagonalu (promienie kół), więc jest macierzą pełnego rzędu, przez co jest również macierzą odwracalną.

Problem pojawia się w dwóch pozostałych przypadkach robotów, gdy układ równań jest nadokreślony, czyli liczba równań jest większa niż wymiar przestrzeni (wektor niewiadomych).

1.6.2 Car-like (Ackermann)

Układ z car-like jak nazwa wskazuje jest podobny do konfiguracji kół w samochodzie. Układ Ackermanna polega na takim sterowaniu skrętem kół podczas jazdy po łuku, aby każde koło poruszało się po okręgu o wspólnym środku (tzw. ICR - chwilowy punkt obrotu). Gwarantuje to ruch bez poślizgu.

Tab. 1.2: Parametry kół robota car-like

Koło	Położenie	Rodzaj	Ogr. toczenia	Ogr. poślizgu
1	lewy przód	skrętne	tak	tak
2	lewy tył	stałe	tak	tak
3	prawy tył	stałe	tak	tak
4	prawy przód	skrętne	tak	tak

$$A = \begin{bmatrix} \sin(\alpha_1 + \beta_1 + \theta_1) & -\cos(\alpha_1 + \beta_1 + \theta_1) & -l_1 \cos(\beta_1 + \theta_1) \\ \sin(\alpha_2 + \beta_2) & -\cos(\alpha_2 + \beta_2) & -l_2 \cos(\beta_2) \\ \sin(\alpha_3 + \beta_3) & -\cos(\alpha_3 + \beta_3) & -l_3 \cos(\beta_3) \\ \sin(\alpha_4 + \beta_4 + \theta_4) & -\cos(\alpha_4 + \beta_4 + \theta_4) & -l_4 \cos(\beta_4 + \theta_4) \\ \cos(\alpha_1 + \beta_1 + \theta_1) & \sin(\alpha_1 + \beta_1 + \theta_1) & l_1 \sin(\beta_1 + \theta_1) \\ \cos(\alpha_2 + \beta_2) & \sin(\alpha_2 + \beta_2) & l_2 \sin(\beta_2) \\ \cos(\alpha_3 + \beta_3) & \sin(\alpha_3 + \beta_3) & l_3 \sin(\beta_3) \\ \cos(\alpha_4 + \beta_4 + \theta_4) & \sin(\alpha_4 + \beta_4 + \theta_4) & l_4 \sin(\beta_4 + \theta_4) \end{bmatrix}$$

gdzie:

- α_n – orientacja koła n w układzie robota,
- β_n – kąt skrętu koła (dla kół stałych $\beta_n = 0$),
- θ_n – obliczony kąt skrętu,
- l_n – odległość koła od środka układu robota.

$$B = \begin{bmatrix} r & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & r & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & r & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & r & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Gdzie:

- r – promień koła.

Dla tego robota nadokreślony układ równań wymaga zastosowania np. metody najmniejszych kwadratów do obliczenia wektora niewiadomych. Zamiast szukać dokładnego rozwiązania, szukamy takiego $\dot{\varphi}$, które minimalizuje błąd:

$$\min_{\dot{\varphi}} \|A \cdot \dot{\xi}_R - B \cdot \dot{\varphi}\|^2$$

Rozwiązaniem tej optymalizacji w sensie najmniejszych kwadratów jest:

$$\dot{\varphi} = (B^T B)^{-1} B^T A \cdot \dot{\xi}_R$$

lub ogólniej:

$$\dot{\varphi} = B^+ A \cdot \dot{\xi}_R$$

gdzie B^+ to pseudoinwersja Moore'a-Penrose'a macierzy B .

Taki sam problem występuje z robotem:

1.6.3 Skid-steer

W tym układzie wszystkie koła są stałe. Realizacja skrętu zawsze wiąże się z poślizgiem bocznym kół, przez co warunki na brak poślizgu naturalnie nie mogą być spełnione (ale są wymagane więc muszą być uwzględnione w macierzy $\mathbf{A}$).

Tab. 1.3: Parametry kół robota skid steer.

Koło	Położenie	Rodzaj	Ogr. toczenia	Ogr. poślizgu
1	lewy przód	stałe	tak	tak
2	lewy tył	stałe	tak	tak
3	prawy tył	stałe	tak	tak
4	prawy przód	stałe	tak	tak

$$A = \begin{bmatrix} \sin(\alpha_1 + \beta_1) & -\cos(\alpha_1 + \beta_1) & -l_1 \cos(\beta_1) \\ \sin(\alpha_2 + \beta_2) & -\cos(\alpha_2 + \beta_2) & -l_2 \cos(\beta_2) \\ \sin(\alpha_3 + \beta_3) & -\cos(\alpha_3 + \beta_3) & -l_3 \cos(\beta_3) \\ \sin(\alpha_4 + \beta_4) & -\cos(\alpha_4 + \beta_4) & -l_4 \cos(\beta_4) \\ \cos(\alpha_1 + \beta_1) & \sin(\alpha_1 + \beta_1) & l_1 \sin(\beta_1) \\ \cos(\alpha_2 + \beta_2) & \sin(\alpha_2 + \beta_2) & l_2 \sin(\beta_2) \\ \cos(\alpha_3 + \beta_3) & \sin(\alpha_3 + \beta_3) & l_3 \sin(\beta_3) \\ \cos(\alpha_4 + \beta_4) & \sin(\alpha_4 + \beta_4) & l_4 \sin(\beta_4) \end{bmatrix}$$

gdzie:

- α_n – orientacja koła n w układzie robota,
- β_n – kąt skrętu koła (dla kół stałych $\beta_n = 0$),
- l_n – odległość koła od środka układu robota.

$$B = \begin{bmatrix} r & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & r & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & r & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & r & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Gdzie:

- r – promień koła.

W robocie typu car-like dzięki temu, że spełniamy wszystkie równania (tak dobieramy kąty skrętu kół aby koła poruszały się po okręgach o wspólnym środku) pomimo wykorzystania metody najmniejszych kwadratów wynik będzie dokładny. W przypadku skid steer, gdzie część równań nie może być spełnionych wynik nie będzie dokładny a zamiast tego będzie tylko przybliżeniem. Jednak w praktyce okazuje się, że optymalne rozwiązanie takiego układu pozwala na osiągnięcie przez robota prędkości bliskich pożądanym.

Należy pamiętać, że w przypadku takich robotów, gdzie występuje poślizg obliczenie odometrii (kinematyka

prosta) będzie obarczone dużym i szybko narastającym błędem.

1.7 Obliczanie kąta skrętu koła (steering angle)

W tej sekcji przedstawiono sposób obliczania kąta skrętu koła na podstawie prędkości translacyjnej i kątowej robota, uwzględniając przypadki czystej translacji, czystego obrotu oraz ich kombinacji.

1.7.1 Czysta translacja

W przypadku czystej translacji (brak obrotu) zadana jest kombinacja prędkości w osiach X i Y. W rezultacie kąt skrętu może być zerowy jeśli zadany wektor prędkości pokrywa się z nominalnym kierunkiem toczenia kół lub jest niezerowy gdy robot porusza się w osi Y lub po skosie. Kąt skrętu jest obliczany jako:

$$\theta = \text{atan2}(v_y, v_x) \quad (1.16)$$

gdzie:

- v_x – prędkość liniowa w osi X,
- v_y – prędkość liniowa w osi Y.

1.7.2 Czysty obrót

W przypadku czystego obrotu (brak translacji), kąt skrętu jest równy:

$$\theta = -\beta \quad (1.17)$$

gdzie β to stały kąt skrętu, który określa orientację kół w danym przypadku obrotu.

1.7.3 Połączenie translacji i obrotu

W przypadku kombinacji translacji i obrotu (np. dla układów Ackermann), kąt skrętu koła wyliczany jest na podstawie prędkości robota i odległości od środka obrotu:

$$\text{wheelVel}_x = v_x - \omega_z \cdot y \quad (1.18)$$

$$\text{wheelVel}_y = v_y + \omega_z \cdot x \quad (1.19)$$

gdzie:

- v_x – prędkość liniowa w osi X,
- v_y – prędkość liniowa w osi Y,
- ω_z – prędkość kątowa wokół osi Z,
- x – odległość środka robota w osi X,
- y – odległość środka robota w osi Y.

Kąt skrętu obliczany jest jako:

$$\theta_{\text{steering}} = \text{atan2}(\text{wheelVel}_y, \text{wheelVel}_x) \quad (1.20)$$

1.7.4 Korekta kąta dla ruchu wstecznego

Aby zachować spójność w przypadku ruchu wstecznego, kąt skrętu jest dostosowywany:

$$\text{if } (v_x < 0) \quad \text{then} \quad \theta_{\text{steering}} + = \pm\pi$$

1.8 Analiza mobilności i sterowalności robota mobilnego

1.8.1 Stopnie swobody robota (Degrees of Freedom, DoF)

Stopnie swobody określają, jakie niezależne ruchy może wykonać robot w swojej przestrzeni operacyjnej. Dla robota mobilnego poruszającego się na płaszczyźnie 2D mogą istnieć trzy stopnie swobody:

- Przesunięcie wzdłuż osi X,
- Przesunięcie wzdłuż osi Y,
- Obrót wokół osi Z (zmiana orientacji).

Nie każdy robot posiada możliwość kontrolowania wszystkich trzech komponentów ruchu.

1.8.2 Mobilność robota (Degree of Mobility, δ_m)

Mobilność określa, ile niezależnych stopni swobody można kontrolować **bezpośrednio** poprzez zmianę prędkości kół. Jest definiowana jako:

$$\delta_m = 3 - \text{rank}(C_1(\beta_s))$$

gdzie:

- $C_1(\beta_s)$: macierz ograniczeń bocznych wynikających z warunku braku poślizgu bocznego.

Interpretacja wartości:

- $\delta_m = 3$: pełna mobilność – robot może poruszać się w x , y i obracać się,
- $\delta_m = 2$: np. robot różnicowy (brak możliwości jazdy bokiem),
- $\delta_m = 1$: np. trójkołowiec z jednym napędzanym kołem,
- $\delta_m = 0$: robot statyczny (brak mobilności).

$\text{rank}(C_1(\beta_s))$ to rząd macierzy zawierającej ograniczenia na brak poślizgu poprzecznego dla wszystkich kół w układzie. Rząd oznacza ilość wektorów liniowo niezależnych w macierzy, czyli czy przekształcenie za pośrednictwem danej macierzy nie powoduje utraty wymiaru. Jeżeli macierz ma rząd = 3, to 3-elementowy wektor prędkości zostaje przekształcony we wszystkich trzech wymiarach na inny wektor 3-elementowy. Jeżeli macierz ma rząd = 2, to oznacza, że tracimy jeden wymiar i wektor 3-elementowy przemnożony przez tę macierz zostaje przekształcony do płaszczyzny.

C_1 to macierz, która zawiera rzutowanie (projekcje) ruchów wszystkich kół na kierunki prostopadłe do

płaszczyzn ich toczenia (czyli na kierunki, w których nie powinny się poruszać – boczny poślizg).

W tym przypadku sprawdzamy rząd macierzy C_1 , czyli im większy rząd, tym więcej ograniczeń kinematycznych.

1.9 Sterowalność robota (Degree of Steerability, δ_s)

Sterowalność oznacza liczbę niezależnych kół skrętnych, które wpływają na zmianę orientacji osi toczących się. Jest definiowana jako:

$$\delta_s = \text{rank}(C_{1s}(\beta_s))$$

- Zmiana kąta koła skrętnego **nie wpływa od razu na pozycję robota** – efekt jest widoczny dopiero **podczas ruchu**.
- Wzrost rangi C_{1s} oznacza, że mamy **więcej niezależnych możliwości sterowania** kierunkiem ruchu $\rightarrow$ **większa manewrowalność**.
- Uwaga: **to samo koło skrętne** może **zmniejszyć mobilność** (dodaje ograniczenie chwilowe), ale **zwiększyć sterowalność** (bo potrafi zmieniać orientację).

1.9.1 Typowe wartości:

- $\delta_s = 0 \rightarrow$ brak kół skrętnych (np. differential drive),
- $\delta_s = 1 \rightarrow$ np. trójkołowiec z jednym skrętnym kołem,
- $\delta_s = 2 \rightarrow$ np. robot z dwoma niezależnymi kołami skrętnymi.

1.10 Manewrowalność robota (Degree of Maneuverability, δ_M)

Całkowita liczba stopni swobody, które można kontrolować (prędkością lub sterowaniem), to:

$$\delta_M = \delta_m + \delta_s$$

Uwaga: Dwa roboty o tej samej manewrowalności δ_M **nie muszą być równoważne** pod względem zachowania.

1.10.1 Na przykład:

Roboty z napędem różnicowym (differential drive) i roboty trójkołowe (tricycle) mają taką samą manewrowalność:

$$\delta_M = 2.$$

Ale różnice są istotne:

- Dla napędu różnicowego:

$$\delta_m = 2, \quad \delta_s = 0 \quad \Rightarrow \quad \text{cała manewrowalność pochodzi z mobilności (napędu)}.$$

— Dla trójkołowca:

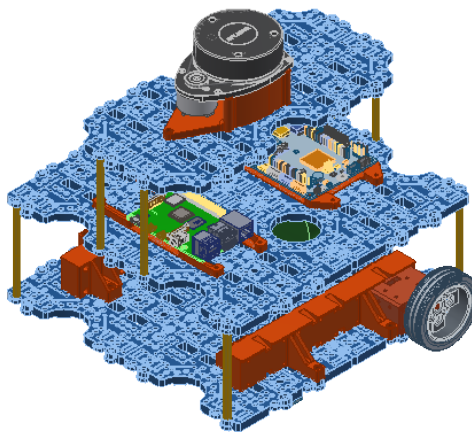
$$\delta_m = 1, \quad \delta_s = 1 \quad \Rightarrow \quad \text{manewrowalność wynika częściowo z napędu, a częściowo z możliwości skrętu.}$$

Wniosek: Mimo tej samej liczby dostępnych stopni swobody (manewrowalności), sposób ich realizacji może być zupełnie inny.

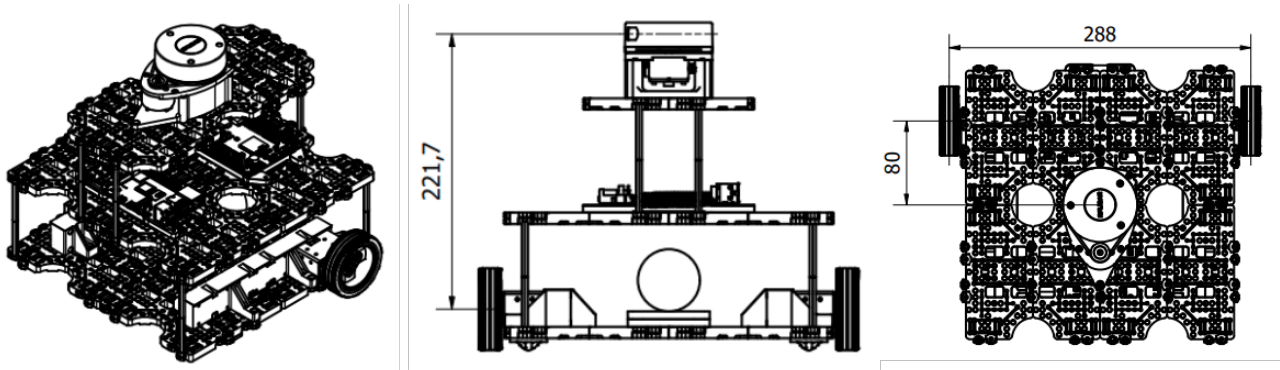
2 Roboty

2.1 Stanowisko 1 - napęd różnicowy

Sterowanie platformą różnicową opiera się na niezależnym napędzaniu dwóch kół po obu stronach robota, co umożliwia precyzyjne manewrowanie poprzez różnicowanie prędkości tych kół. Aby platforma poruszała się na wprost, oba koła muszą obracać się z taką samą prędkością i w tym samym kierunku. Z kolei różnica w prędkościach kół powoduje skręt — im większa różnica, tym ostrzejszy zakręt, a jeśli jedno koło obraca się do przodu, a drugie do tyłu, platforma wykonuje pełny obrót wokół własnej osi. Sterowanie platformą różnicową można realizować poprzez prostą regulację prędkości obrotu każdego koła, co czyni ją łatwą do implementacji w systemach autonomicznych i zdalnie sterowanych. W praktyce, sterowanie różnicowe jest szczególnie skuteczne w aplikacjach, które wymagają dużej manewrowości, takich jak poruszanie się w ograniczonych przestrzeniach. Algorytmy sterujące mogą dynamicznie dostosowywać prędkości kół na podstawie sygnałów z czujników, aby precyzyjnie kontrolować kierunek i trajektorię ruchu robota, umożliwiając omijanie przeszkód i dokładne nawigowanie w wyznaczonym obszarze.



Rys. 2.1: Rzut izometryczny robota mobilnego z napędem różnicowym.

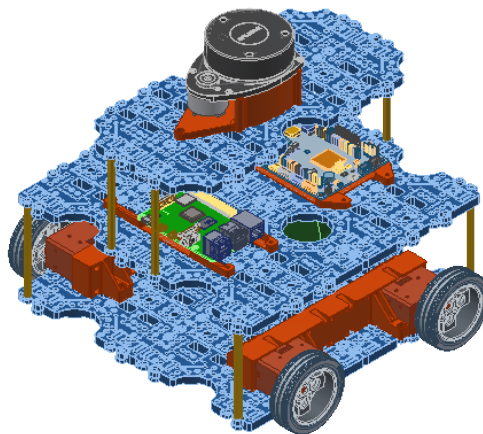


Rys. 2.2: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

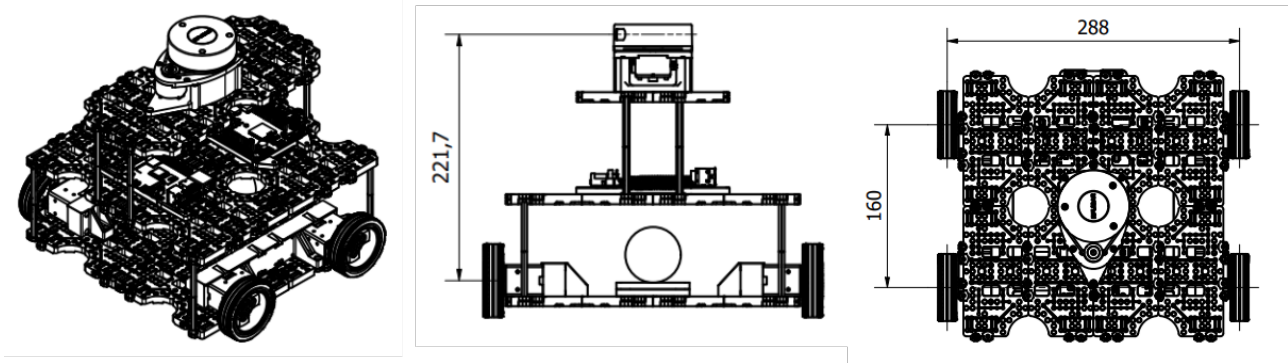
2.2 Stanowisko 2 - napęd skid-steer

Napęd skid-steer, stosowany w pojazdach takich jak ładowarki czy roboty terenowe, opiera się na napędzaniu kół lub gąsienic po obu stronach pojazdu. Sterowanie polega na różnicowaniu prędkości między lewą i prawą stroną – kiedy koła po jednej stronie obracają się szybciej, pojazd skręca w stronę wolniejszej strony, a przy obrocie kół w przeciwnych kierunkach wykonuje obrót w miejscu. W przeciwieństwie do platformy różnicowej, w napędzie skid-steer wszystkie koła są sztywno zamocowane i ustawione równolegle, co oznacza, że do wykonania skrętu pojazd musi wymusić poślizg kół.

Ten poślizg jest niezbędny dla skrętu, lecz wprowadza istotne ograniczenie – zasada braku poślizgu nie jest tu możliwa do spełnienia. W efekcie poślizg kół powoduje błędy w odometrii, ponieważ przemieszczenie platformy nie odpowiada dokładnie obrotom kół, szczególnie podczas skręcania. Odczyty odometryczne zawierają błędy, co sprawia, że pojazd może nie precyzyjnie odwzorować zamierzoną trasę bez dodatkowych korekt. W praktyce oznacza to konieczność stosowania dodatkowych sensorów, takich jak IMU, GPS czy LiDAR, aby kompensować odchylenia wynikające z poślizgu i utrzymać dokładność nawigacji. Napęd skid-steer jest wytrzymały i dobrze sprawdza się na trudnym terenie, gdzie poślizg jest naturalny, lecz jego precyzja odometrii na równym podłożu jest niższa niż w przypadku klasycznego napędu różnicowego.



Rys. 2.3: Rzut izometryczny robota mobilnego z napędem różnicowym typu skid-steer.



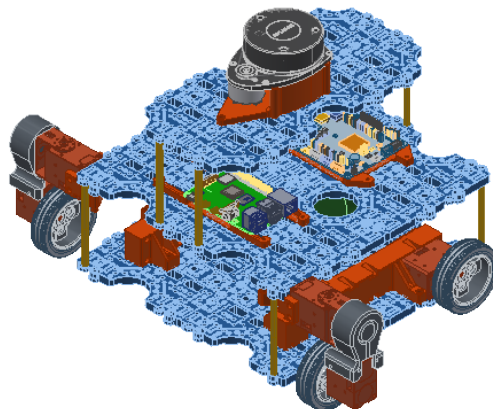
Rys. 2.4: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.3 Stanowisko 3 - napęd car-like

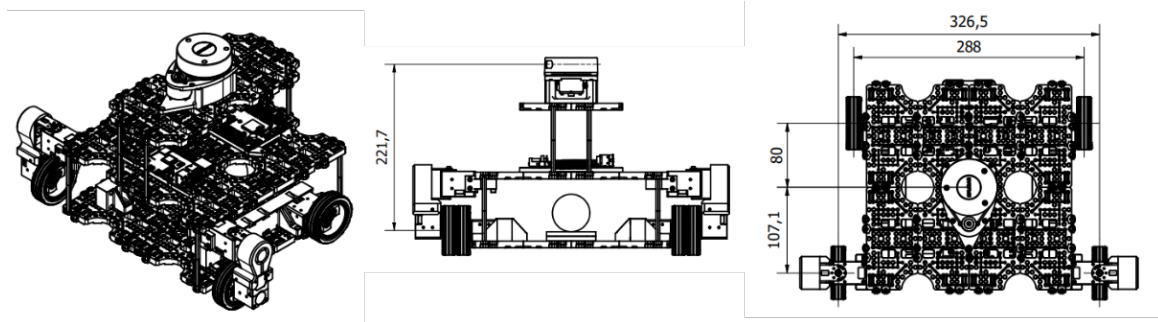
Napęd car-like, czyli napęd Ackermana, stosowany jest w pojazdach takich jak samochody czy niektóre roboty mobilne. Charakteryzuje się tym, że skręcanie odbywa się przez zmianę kąta obrotu przednich kół, podczas gdy tylne koła pozostają skierowane na wprost. W przeciwieństwie do napędów różnicowego czy skid-steer, napęd Ackermana nie wymusza poślizgu kół – każde koło podąża za odpowiednią krzywizną podczas skrętu.

Sterowanie takim układem polega na precyzyjnym ustawieniu kątów przednich kół tak, aby punkt przecięcia ich osi pokrywał się z punktem obrotu pojazdu.

Jednak napęd Ackermana ma swoje ograniczenia – promień skrętu zależy od maksymalnego kąta wychylenia przednich kół, co ogranicza manewrowość w ciasnych przestrzeniach w porównaniu do platform różnicowych czy skid-steer, które mogą obracać się w miejscu. Tego typu napęd jest idealny do zastosowań, gdzie wymagana jest stabilność i przewidywalność ruchu, takich jak autonomiczne samochody lub roboty poruszające się po utwardzonych nawierzchniach.



Rys. 2.5: Rzut izometryczny robota mobilnego z napędem Ackermana.



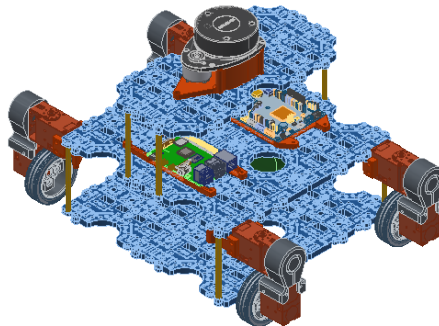
Rys. 2.6: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.4 Stanowisko 4 - napęd omni (swerve drive)

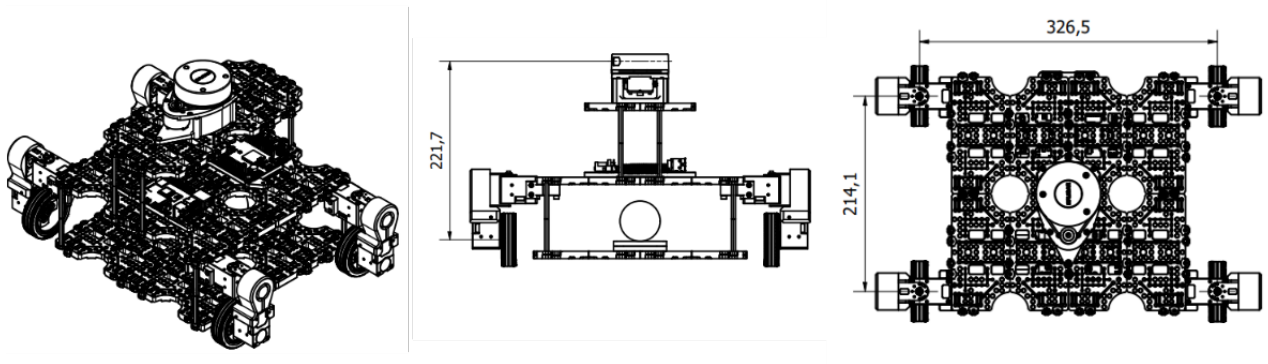
Napęd typu swerve-drive to zaawansowany system stosowany w robotach mobilnych, który zapewnia wyjątkową manewrowość i swobodę ruchu. Każde koło jest zamocowane na własnej osi obrotu, co oznacza, że może niezależnie zmieniać zarówno kierunek, jak i prędkość. Dzięki temu każde z kół może być skierowane w dowolnym kierunku, co umożliwia płynne i precyzyjne poruszanie się robota we wszystkich kierunkach – także na boki i po skosie (omnidrive). W przeciwieństwie do innych napędów, takich jak różnicowy czy skid-steer, napęd swerve pozwala na natychmiastową zmianę kierunku ruchu bez konieczności zmiany orientacji całego robota.

Sterowanie takim napędem wymaga zaawansowanej kontroli, ponieważ zarówno kąt skrętu, jak i prędkość każdego z kół muszą być niezależnie regulowane. Dzięki tej precyzyjnej kontroli robot z napędem swerve może poruszać się po złożonych trajektoriach, na przykład omijając przeszkody w sposób ciągły lub precyzyjnie docierając do wyznaczonego punktu bez zmiany swojej orientacji. Jest to szczególnie przydatne w środowiskach, gdzie wymagana jest duża manewrowość i szybkie dostosowywanie kierunku ruchu, na przykład w magazynach, czy przy liniach produkcyjnych.

W napędzie swerve nie występuje poślizg kół jako konieczny element manewrowania, co oznacza, że ruch jest płynny i kontrolowany. Dzięki temu odometria robota – czyli obliczanie pozycji na podstawie ruchu kół – jest dokładniejsza niż w przypadku napędu skid-steer, gdzie poślizg powoduje błędy. To rozwiązanie sprawdza się najlepiej tam, gdzie potrzebna jest doskonała manewrowość i elastyczność, a ograniczenia przestrzeni wymagają szybkich i precyzyjnych manewrów.



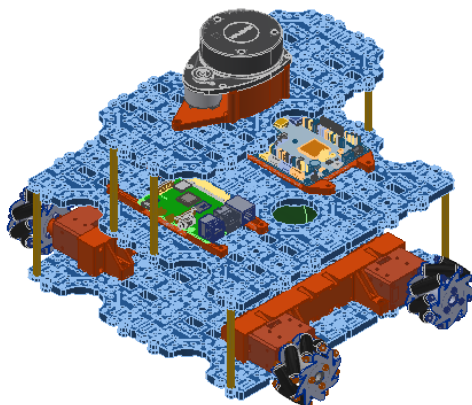
Rys. 2.7: Rzut izometryczny robota mobilnego z napędem swerve-drive.



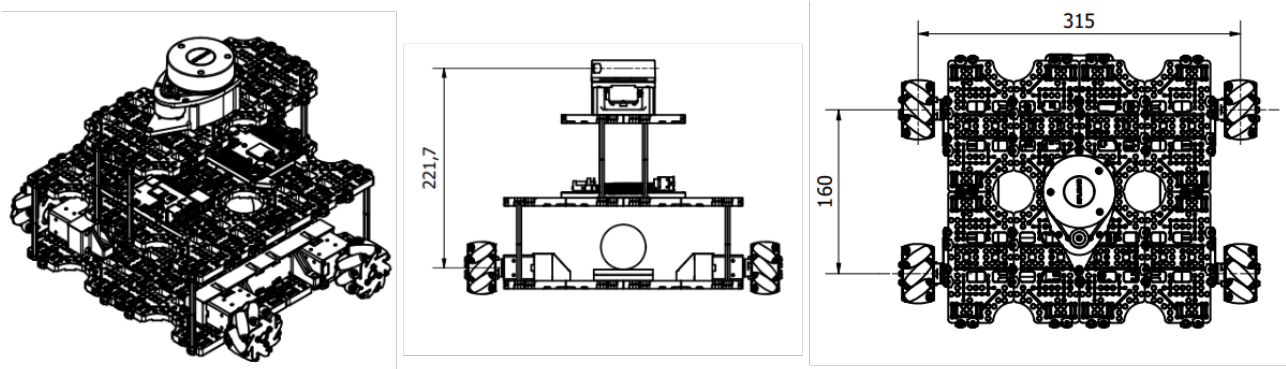
Rys. 2.8: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.5 Stanowisko 5 - napęd omni (mecanum)

Napęd typu mecanum to system umożliwiający pełną swobodę ruchu robota w dowolnym kierunku dzięki zastosowaniu specjalnych kół mecanum. Każde koło mecanum jest wyposażone w rolki ustawione pod kątem 45 stopni, co pozwala robotowi poruszać się w dowolnym kierunku bez zmiany orientacji. Sterowanie napędem mecanum polega na niezależnym kontrolowaniu prędkości obrotu każdego koła. Kiedy koła obracają się w określonym układzie, prędkości wynikowe rolek dodają się, aby wywołać ruch w pożądanym kierunku. Na przykład, aby poruszać się do przodu, wszystkie koła obracają się w tym samym kierunku, natomiast aby poruszać się na boki, koła obracają się w przeciwnych kierunkach. Możliwość niezależnego sterowania każdym kołem umożliwia wykonywanie skomplikowanych manewrów, takich jak ruch po przekątnej czy obrót w miejscu. Ograniczeniem kół mecanum jest zmniejszona wydajność na niektórych rodzajach terenu. Chociaż koła mecanum sprawdzają się dobrze na płaskich, gładkich powierzchniach, mogą mieć trudności na nierównym lub szorstkim terenie, gdzie rolki mogą tracić kontakt z podłożem, co prowadzi do zmniejszenia trakcji i kontroli.



Rys. 2.9: Rzut izometryczny robota mobilnego z napędem mecanum.



Rys. 2.10: Wymiary konstrukcji niezbędne do zaimplementowania modelu kinematyki.

2.6 Uruchomienie robota

1. Umieść naładowaną baterię w robocie i przełącz przełącznik zasilania na niebieskiej płytce OpenCR na pozycję "ON".
2. Robot zacznie się uruchamiać. Należy poczekać ok. 1 minuty do pojawienia się robota w sieci.
3. (Na komputerze) Połącz się z siecią wifi laboratorium (SSID: RM_LAB*)
4. (Na komputerze) Przejdź w terminalu do właściwej lokalizacji

/home/student/Pulpit/materiały_do_zajec/ros_fun-lab_roboty_mobilne

Można to zrobić na dwa sposoby:

- (a) Przejsz z widoku folderu do katalogu a następnie kliknąć prawym przyciskiem myszki w widoku folderu i wybrać opcję otwórz w terminalu.
- (b) Uruchomić terminal w dowolnej lokalizacji a następnie wpisać:

```
cd /home/student/Pulpit/materiały\_do\_zajec/ros\_fun-lab\_roboty\_mobilne/
```

5. W terminalu z właściwą lokalizacją (ustawioną zgodnie z poprzednim punktem) należy uruchomić kontener:

```
git add docker-compose-x11.yml
```

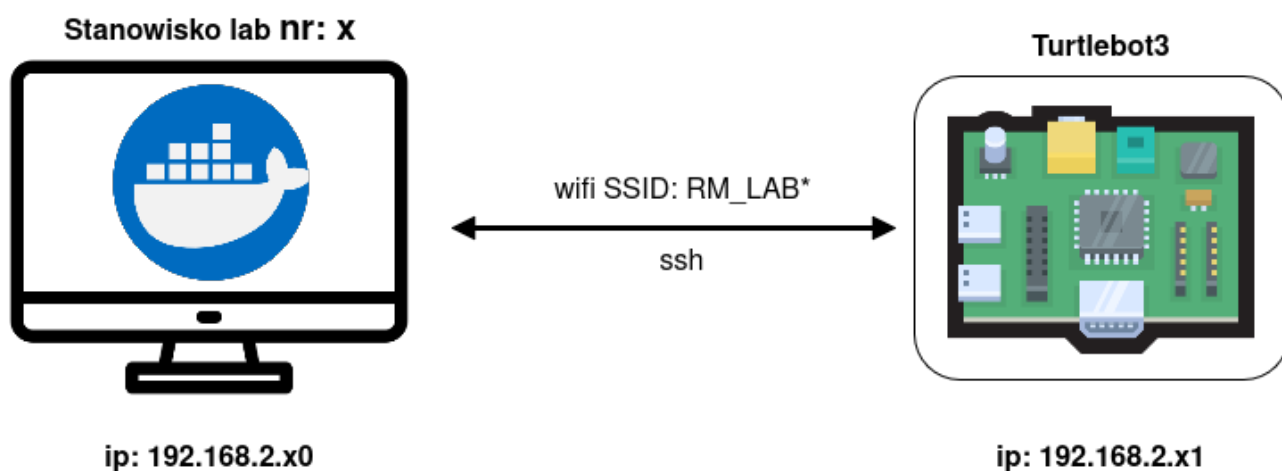
6. Po załadowaniu kontenera jego środowisko graficzne jest dostępne za pośrednictwem przeglądarki na porcie:

```
localhost:6080
```

7. Uruchom terminal w środowisku docker.
8. Za pomocą ssh połącz się z robotem

```
ssh pi@192.168.2.x1
```

gdzie x to numer stanowiska. Hasło to "ubuntu".



Rys. 2.11: Schemat komunikacji z robotem.

Urządzenie	IP	ROS_DOMAIN_ID
Główny router Teltonika RUTM10	192.168.2.1	
PC stanowisko 1	192.168.2.10	10
PC stanowisko 2	192.168.2.20	20
PC stanowisko 3	192.168.2.30	30
PC stanowisko 4	192.168.2.40	40
PC stanowisko 5	192.168.2.50	50
TB3_1 RPi4 (2 wheel diff)	192.168.2.11	10
TB3_2 RPi4 (skid-steer 4 wheels)	192.168.2.21	20
TB3_3 RPi4 (car like 4 wheels)	192.168.2.31	30
TB3_4 RPi4 (swerve drive 4 wheels)	192.168.2.41	40
TB3_5 RPi4 (mecanum 4 wheels)	192.168.2.51	50
TB4_1 RPi4 (Livox 360)	192.168.2.12	10
TB4_2 RPi4 (OAK camera)	192.168.2.22	20
TB4_3 RPi4	192.168.2.32	30
TB4_4 RPi4	192.168.2.42	40
TB4_5 RPi4	192.168.2.52	50

Rys. 2.12: Konfiguracja sieci w laboratorium.

9. Punkt wykonać zarówno w środowisku docker jak i na robocie:

Otwórz plik `.bashrc`. Jest to plik ładowany przy uruchomieniu każdej sesji terminala. Z poziomu terminala należy wpisać

```
nano .bashrc
```

W terminalu powinien uruchomić się edytor umożliwiający modyfikację pliku. Na końcu pliku powinna znajdować się linia z `ROS_DOMAIN_ID`, którą należy odkomentować (usunąć znak `#`) lub dopisać, jeśli nie istnieje. Linia powinna wyglądać następująco:

```
export ROS_DOMAIN_ID=30
```

Gdzie w miejscu numer ID należy wpisać zgodny z tabelą powyżej. Następnie należy wyjść z terminala zapisując zmiany (`ctrl+x`). Aby zmiany zaczęły działać należy załadować plik ponownie komendą:

```
source ~/.bashrc
```

10. W środowisku pi wywołaj komendę:

```
ros2 launch turtlebot3_bringup RM_lab.launch.py
```

Spowoduje to włączenie odpowiednich nodów na robocie (komunikacja z czujnikami, sterowanie napędami).

2.7 Uruchomienie środowiska Jupyter Notebook

Jupyter Notebook to interaktywne środowisko do uruchamiania kodu, które pozwala na pisanie, wykonywanie i dokumentowanie obliczeń w jednym pliku. W ramach laboratorium wykorzystywane będzie z językiem Python3.

Kluczowe cechy Jupyter Notebook:

- Komórki kodu – umożliwiają uruchamianie fragmentów kodu niezależnie od siebie.
- Komórki tekstowe (Markdown) – pozwalają na dodawanie opisów, równań LaTeX oraz formatowanego tekstu.
- Interaktywność – wyniki obliczeń, wykresy i dane mogą być wyświetlane bezpośrednio w notebooku.

W celu uruchomienia środowiska:

1. Wywołaj w terminalu w środowisku docker komendę:

```
jupyter-notebook
```

Środowisko uruchomi się na porcie 8888 w przeglądarce w kontenerze. Dla większej wygody można skopiować adres

```
http://0.0.0.0:8888/tree
```

i wkleić do przeglądarki na systemie komputera w laboratorium.

2.8 Sterowanie robotem

W celu zadania prędkości do robota należy wysłać wiadomość typu *geometry_msgs/Twist* na topic o nazwie */cmd_vel*. Struktura wiadomości *geometry_msgs/Twist*.

```

geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z

```

gdzie w `linear.x` i `linear.y` należy podać zadaną prędkość liniową w $\frac{m}{s}$ a w `angular.z` zadaną prędkość obrotową w $\frac{rad}{s}$.

Wysłać wiadomość na topic `/cmd_vel` można na kilka sposobów:

- Ręcznie w oknie terminala wykorzystując komendę `ros2 topic pub`, np:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "linear: x: 0.5, y: 0.0, z: 0.0, angular: x: 0.0, y: 0.0, z: 0.2"
```

```

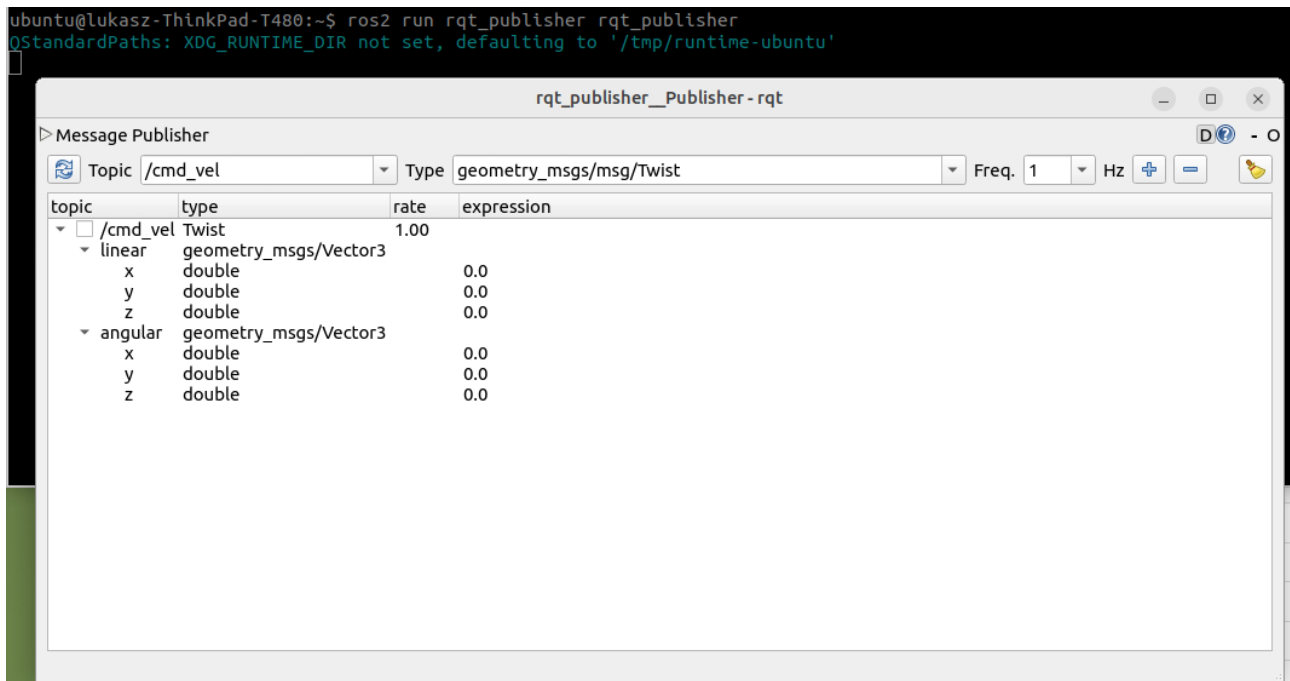
ubuntu@lukasz-ThinkPad-T480:~$ ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.5, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.2}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
^Cubuntu@lukasz-ThinkPad-T480:~$

```

Rys. 2.13: Sterowanie z terminala.

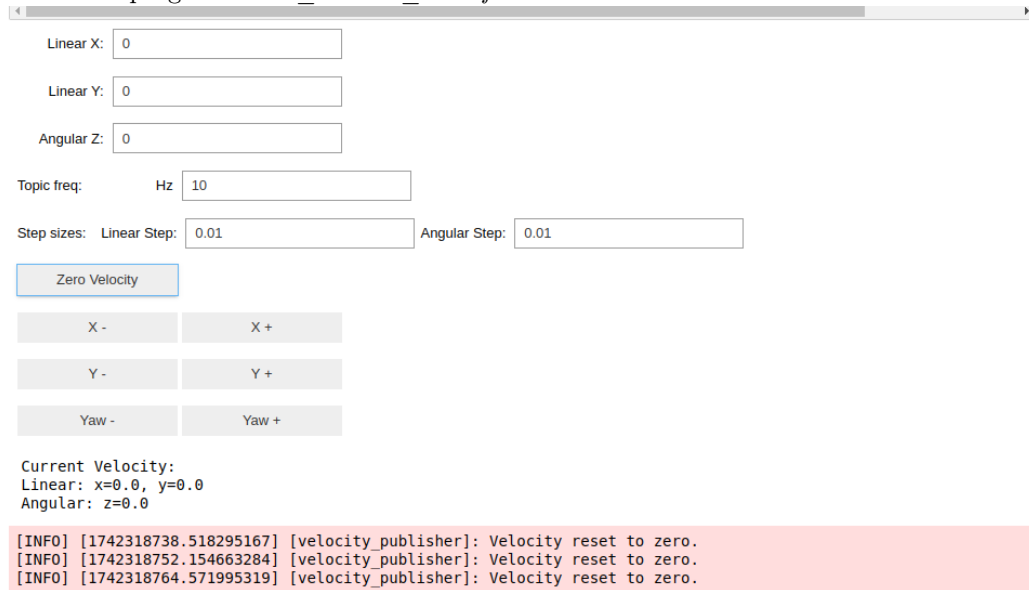
- Wykorzystując graficzny interfejs `rqt_publisher`. W tym celu uruchom program za pomocą komendy:

```
ros2 run rqt_publisher rqt\_publisher
```



Rys. 2.14: Sterowanie z `rqt_publisher`.

- Wykorzystując graficzny interfejs przygotowany w jupyter notebook. W tym celu z katalogu z instrukcjami do zajęć uruchom program `robot_control_interface`.



Rys. 2.15: Sterowanie z jupyter notebook.

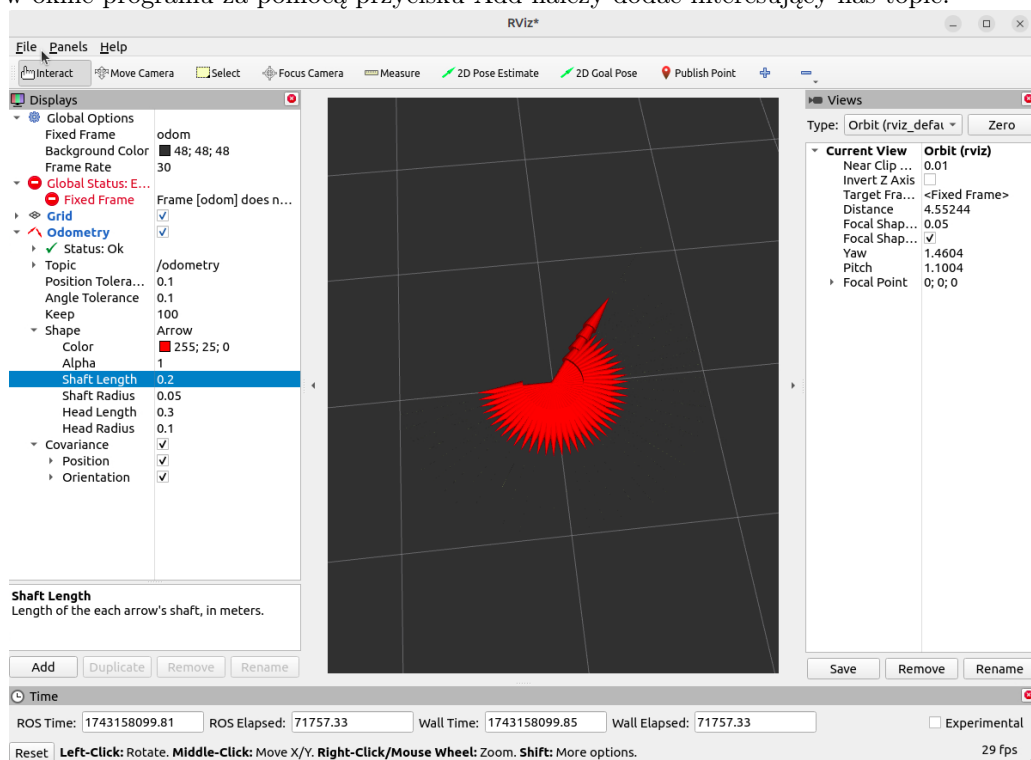
2.9 Wizualizowanie wyników z rviz2

Program rviz2 służy do wizualizowania danych w różnych formatach w charakterystyczny dla nich sposób. Można w ten sposób m.in. zbadać poprawność generowania trajektorii robota.

W celu uruchomienia programu rviz2 wystarczy w terminalu wpisać:

```
rviz2
```

Następnie w oknie programu za pomocą przycisku Add należy dodać interesujący nas topic.



Rys. 2.16: Wizualizacja danych za pomocą rviz2.

3 Program ćwiczenia

Ćwiczenie 1 - Sprawdzenie stopnia mobilności robota

Zadanie:

Oblicz stopień mobilności δ_m swojego robota, korzystając z macierzy $C_1(\beta_s)$. W tym celu wykonaj poniższe kroki:

- Zdefiniuj odpowiednią macierz C_1 , zawierającą ograniczenia kinematyczne wynikające z warunku braku poślizgu bocznego.
- Oblicz rząd macierzy C_1 i na tej podstawie wyznacz mobilność robota.
- Wykonaj obliczenia w jupyter notebook.

Podpowiedź:

- Użyj funkcji `np.linalg.matrix_rank()` do obliczenia rzędu macierzy C_1 .
- Zgodnie z definicją $\delta_m = 3 - \text{rank}(C_1)$.

Oczekiwany wynik:

Po obliczeniu rzędu macierzy, otrzymasz wartość δ_m , która odpowiada za stopień mobilności robota.

Ćwiczenie 2 - Zaprogramowanie funkcjonalności kinematyki odwrotnej

Zadanie:

Zaprogramuj funkcję `cmd_vel_callback()` aby obliczała prędkości kół na podstawie zadanych prędkości robota.

- Funkcja powinna:
 - Odbierać wektor prędkości robota (w formacie `Twist`).
 - Obliczać prędkości kół przy pomocy macierzy kinematyki odwrotnej.
 - Publikować obliczone prędkości kół na temat `/cmd_joint_states`.

Podpowiedź:

- Skorzystaj z macierzy A (ograniczenia kinematyczne) oraz B (transformacja prędkości kół).
- Wykorzystaj wzór na obliczanie prędkości kół:

$$\dot{\varphi} = (B^T B)^{-1} B^T A \dot{\xi}_R$$

Ćwiczenie 3 - Obliczanie prędkości robota w kinematyce prostej

Zadanie:

Napisz funkcję `joint_states_callback()` obliczającą prędkości robota na podstawie prędkości kół i przekształć ją w prędkość robota.

- Funkcja powinna:
 - Odbierać dane o prędkościach kół (w formacie `JointState`).
 - Obliczać wektor prędkości robota w przestrzeni kartezjańskiej.

- Na podstawie tego wektora aktualizować pozycję robota na płaszczyźnie.
- Publikować obliczoną odometrię robota na temat /odometry.

Podpowiedź:

- Skorzystaj z równań kinematyki prostej:

$$\dot{\xi}_R = (A^T A)^{-1} A^T B \dot{\varphi}$$

- Oblicz aktualną pozycję robota przy pomocy zmiennych x , y , θ oraz oblicz prędkości robota.

Ćwiczenie 4 - Weryfikacja wyników na rzeczywistym robocie

Zadanie:

Przeprowadź testy na rzeczywistym robocie, aby zweryfikować poprawność wyników uzyskanych w poprzednich ćwiczeniach.

- Sprawdź, czy obliczone prędkości kół oraz pozycje robota są zgodne z oczekiwaniami w rzeczywistym świecie.
- Wykonaj testy poruszając robotem w różnych kierunkach oraz zmieniając prędkości kół.